Academia\BackEnd\Academia Parent\Academia_Parents.c

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
*/

#include "Academia_Parents.h"
#include "../Database.h"

#include <stdio.h>
#include <string.h>

#ifndef _WIN32
#include <sqlite3.h>
#endif

/* write_list runs one query and writes the rows as a JS array */
static void write_list(FILE *out, sqlite3 *db, const char *name, const char *sql, int parent_id) {
    sqlite3_stmt *row = NULL;

    fprintf(out, "  %s: [\n", name);

    /* if the query will not prepare, close the array so the page still parses */
    if (sqlite3_prepare_v2(db, sql, -1, &row, NULL) != SQLITE_OK) {
        fputs("  ],\n", out);
        return;
    }
    /* bind the id so the query can never be injected */
    sqlite3_bind_int(row, 1, parent_id);

    /* one line of JavaScript per row */
    while (sqlite3_step(row) == SQLITE_ROW) {
        int i;
        fputs("    {", out);
        for (i = 0; i < sqlite3_column_count(row); i++) {
            const char *text = (const char *)sqlite3_column_text(row, i);

            fprintf(out, "%s%s: ", i ? ", " : "", sqlite3_column_name(row, i));

            /* numbers go in as numbers, everything else as a quoted string */
            if (sqlite3_column_type(row, i) == SQLITE_FLOAT) {
                fprintf(out, "%.1f", sqlite3_column_double(row, i));
            } else if (sqlite3_column_type(row, i) == SQLITE_INTEGER) {
                fprintf(out, "%d", sqlite3_column_int(row, i));
            } else {
                fprintf(out, "\"%s\"", text == NULL ? "" : text);
```

```
            }
        }
        fputs(" },\n", out);
    }

    sqlite3_finalize(row);
    fputs("  ],\n", out);
}

/* update_parent_frontend_state Called by the server for GET /api/parent-data (role "par */
int update_parent_frontend_state(int parent_user_id) {
    sqlite3 *db = open_database();
    sqlite3_stmt *row = NULL;
    /* found stays 0 until the parent row is seen */
    FILE *out = NULL;
    int found = 0;

    if (!db) {
        printf("[Parent] Cannot open the database.\n");
        return 0;
    }

    /* the portal reads this file */
    out = fopen("Academia/FrontEnd/JS/parents_states.js", "w");
    if (!out) {
        printf("[Parent] Cannot write parents_states.js\n");
        close_database(db);
        return 0;
    }

    fputs("// Written by the Academia C backend.\nconst parentState = {\n", out);

    /* the profile block is written column by column */

    /* parent profile */
    fputs("  parent: {\n", out);
    if (sqlite3_prepare_v2(db,
            "SELECT user.username AS username, user.first_name AS firstName, "
            "user.last_name AS lastName, parent.phone AS phone FROM user "
            "INNER JOIN parent ON user.id = parent.id WHERE user.id = ?;",
            -1, &row, NULL) == SQLITE_OK) {
        sqlite3_bind_int(row, 1, parent_user_id);
        if (sqlite3_step(row) == SQLITE_ROW) {
            int i, n = sqlite3_column_count(row);
            found = 1;
            fprintf(out, "    userId: %d,\n", parent_user_id);
            for (i = 0; i < n; i++) {
                const char *text = (const char *)sqlite3_column_text(row, i);
                fprintf(out, "    %s: \"%s\"%s\n", sqlite3_column_name(row, i),
                        text == NULL ? "" : text, i == n - 1 ? "" : ",");
```

```
            }
        }
        sqlite3_finalize(row);
    }
    fputs("  },\n", out);

    /* linked children */
    /* the children come from student.parent_id */
    write_list(out, db, "children",
        "SELECT user.first_name || ' ' || user.last_name AS name, "
        "user.username AS username, student.grade_level AS gradeLevel, "
        "student.gpa AS gpa FROM student INNER JOIN user ON student.id = user.id "
        "WHERE student.parent_id = ? ORDER BY user.id;", parent_user_id);

    /* children's grades */
    write_list(out, db, "grades",
        "SELECT user.first_name || ' ' || user.last_name AS student, "
        "course.course_name AS course, grade.score AS score FROM grade "
        "INNER JOIN student ON grade.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "INNER JOIN course ON grade.course_id = course.id "
        "WHERE student.parent_id = ? ORDER BY user.id, course.id;", parent_user_id);

    /* children's attendance */
    write_list(out, db, "attendance",
        "SELECT user.first_name || ' ' || user.last_name AS student, "
        "course.course_name AS course, attendance.present AS present FROM attendance "
        "INNER JOIN student ON attendance.student_id = student.id "
        "INNER JOIN user ON student.id = user.id "
        "INNER JOIN course ON attendance.course_id = course.id "
        "WHERE student.parent_id = ? ORDER BY user.id, course.id;", parent_user_id);

    /* announcements (school wide) */
    write_list(out, db, "announcements",
        "SELECT announcement.title AS title, announcement.message AS message, "
        "announcement.announcement_date AS date FROM announcement "
        "ORDER BY announcement.id DESC;", 0);

    /* the page checks this flag */
    fprintf(out, "  success: %s,\n  parentId: %d\n};\n",
            found ? "true" : "false", parent_user_id);

    /* done, close both */
    fclose(out);
    close_database(db);
    printf("[Parent] parents_states.js written for parent %d\n", parent_user_id);
    return found;
}
```